

MIMIC-IV QA Benchmark: Implementation Review

Pipeline Phases, Design Choices, and Intermediate Outputs

Fabrizio Pagnozzi
MSc Thesis — Politecnico di Milano

April 2026

Abstract

This document walks through the four pipeline phases used to build the MIMIC-IV QA benchmark. For each phase it explains the rationale behind the key design choices in the context of the thesis argument (coverage vs. dispersion), describes what each step produces, and notes how downstream phases consume it.

Contents

1	Phase 1 — Corpus Construction	2
1.1	1a. Condition selection	2
1.2	1b. Note chunking	2
1.3	1c. Admission metadata	3
1.4	1d. Boilerplate deduplication	3
2	Phase 2 — Embeddings	3
3	Phase 3 — Query Generation and Annotation	3
3.1	3a. Building grounded LLM prompts	3
3.2	3b. LLM query generation	4
3.3	3c. Divergence pre-filter	4
3.4	3d. Gold annotation (map-reduce LLM)	4
4	Phase 4 — Evaluation	5
4.1	Candidate pool construction	5
4.2	Retrieval strategies	5
4.3	Metrics	5

Phase 1 — Corpus Construction

1a. Condition selection

The benchmark is designed to test whether coverage-based retrieval outperforms dispersion when candidate pools have a multi-cluster structure. The first design decision is therefore which conditions to include: we want conditions whose patient population naturally produces that structure, i.e. patients whose discharge notes span multiple distinct clinical problems.

We select conditions by grouping ICD-10 codes at the three-character prefix level (e.g. I50 = heart failure) and ranking by $N_{\text{admissions}} \times \bar{c}^2$, where $\bar{c}$ is the mean Charlson comorbidity count per admission. The quadratic weight on comorbidity richness is deliberate: a patient with five active comorbidities will produce a discharge note covering five distinct clinical problems, each a potential cluster in embedding space. Conditions with flat, single-problem profiles would generate uniform candidate pools where coverage and top- k behave identically. A minimum of 100 admissions is required for statistical reliability.

Output. `conditions_stats.parquet`: ranked condition table with ICD prefix, name, admission count, mean comorbidity count, and composite score. The top N conditions (configurable) feed all downstream phases.

1b. Note chunking

The central thesis claim is about *candidate pool structure*. The granularity of chunks determines that structure: if we chunk at the note level, all of a patient’s clinical problems collapse into one vector. Instead, we aim for one chunk per clinical problem so that each active issue can occupy a distinct region of embedding space.

MIMIC-IV discharge notes come in two formats: a tagged-section structured field and a free-text field. We parse both. Section selection follows a QA-value judgment informed by the structure of discharge summaries:

- **Kept as chunks:** Brief Hospital Course (BHC), History of Present Illness, Pertinent Results, Discharge Diagnosis. These contain actual clinical reasoning and management decisions.
- **Kept as metadata only:** Chief Complaint — too short to stand alone, but useful in the contextual embedding prefix.
- **Skipped:** low-information or deidentified sections (Attending, Social History, Followup Instructions, etc.).

The BHC gets the most careful treatment because it is the richest source. Complex admissions structure it as a problem list delimited by # markers (e.g. # ASCITES, # HEPATIC ENCEPHALOPATHY), so we split on those to produce one chunk per active clinical problem. This directly maps the BHC’s internal structure to the multi-cluster structure we need in the embedding space. Simple or surgical admissions have no # markers and produce a single BHC chunk — analogous to single-hop queries, where coverage is not expected to add anything. The Transitional Issues sub-block (post-discharge follow-up items, not clinical reasoning) is stripped before splitting. Sections that exceed the embedding model’s token limit are split into overlapping windows.

Output. `chunks.parquet`: one row per chunk with a globally unique `chunk_id`, the originating `note_id` and `hadm_id`, `section_name`, and the chunk text. This is the primary corpus artifact that all subsequent phases operate on.

1c. Admission metadata

A chunk like “insulin held, CKD worsening” is patient-specific and will not semantically match a condition-level query without additional context. We build a per-admission metadata table collecting demographics (age, sex, race), admission type, the primary ICD-10 diagnosis, the top-5 diagnoses in human-readable form, and one binary flag per Charlson comorbidity category. This table serves two purposes: (1) constructing the contextual embedding prefix in Phase 2, and (2) filtering candidate admissions by condition and modifier in Phase 3.

Output. `admissions_metadata.parquet`: one row per admission with demographics, diagnosis descriptions, and Charlson comorbidity indicators.

1d. Boilerplate deduplication

Sections like Past Medical History, Allergies, and Family History are often copy-pasted verbatim across a patient’s multiple hospitalizations. Without deduplication these near-identical chunks would form an artificially inflated cluster in embedding space, and any retrieval method would trivially “cover” it. This would produce spuriously high recall independent of the method’s quality. We keep one copy per (patient, section, content hash), retaining the most recent version. Clinical sections (BHC problem entries, assessment) are left untouched, since longitudinal changes there are meaningful evidence.

Output. `chunks.parquet` updated in place, with boilerplate duplicates removed.

Phase 2 — Embeddings

Raw chunks are too patient-specific to match condition-level queries. Following the Contextual Retrieval approach (Anthropic, 2024 — reported 49% reduction in retrieval failure), we prepend a structured natural-language prefix to each chunk before embedding:

*[age group] [sex] ([race]) admitted for [primary diagnosis].
Chief complaint: [text]. Comorbidities: [top ICD descriptions].
Section: [section name].*

This bridges patient-specific narrative to condition-level semantics. Critically, the prefix encodes comorbidity and demographic information, so that modifier-specific queries (e.g. “management of heart failure in elderly patients”) can retrieve appropriately conditioned chunks rather than generic ones. Unlike the original Contextual Retrieval method which uses an LLM to generate the prefix from surrounding context, we use a deterministic template because MIMIC-IV metadata is already structured in relational tables.

Output. LanceDB vector table `chunks`: one row per chunk with all metadata plus a dense float32 embedding vector. At retrieval time the full corpus vectors are loaded into memory as a matrix for fast cosine similarity computation.

Phase 3 — Query Generation and Annotation

3a. Building grounded LLM prompts

The queries must be *multi-aspect* by construction: answering them should require synthesizing information from multiple patients, covering several distinct clinical facets. A textbook question (“what are the symptoms of heart failure?”) would have a single-cluster answer set where coverage adds nothing over top- k .

We generate one query per (condition, modifier) pair. Modifiers are of two types:

- **Comorbidity modifiers:** the top-3 most prevalent Charlson co-occurring conditions for each ICD group, computed dynamically from the data. These are the axes that produce multi-cluster pools: patients with the condition alone form one cluster; patients with both condition and modifier form another.
- **Demographic modifiers:** fixed age-group thresholds (e.g. elderly > 75, young adult < 40). These introduce a different kind of heterogeneity in management patterns.

Rather than prompting the LLM blindly, we inject real BHC and HPI excerpts sampled from actual patients at the condition+modifier intersection. This grounds the LLM in the actual clinical language of the data and steers it toward questions that are answerable from the notes rather than from a medical textbook. The prompt template explicitly requires the question to *connect at least two clinical aspects* (e.g. a drug adjustment and the lab finding that triggered it), ensuring the gold annotation will produce a multi-facet structure.

Output. `queries_prompts.parquet`: one row per (condition, modifier) pair with the condition name, modifier text and type, the IDs of patients used for grounding, and the full assembled prompt.

3b. LLM query generation

Each grounded prompt is sent to a local LLM to produce the clinical question text. The LLM is instructed to return only the question.

Output. `queries.parquet`: the raw benchmark query set with one question per (condition, modifier) pair.

3c. Divergence pre-filter

Not every generated query is useful. A query where facility-location selects nearly the same set of chunks as top- k provides no discriminative signal for the thesis comparison. We keep only queries where coverage *meaningfully diverges*. For each query we run both top- k and facility-location on a cosine top- N pool from the full corpus (the same construction used at evaluation time) and compute:

- **Jaccard divergence** $1 - |S_{FL} \cap S_{top-k}| / |S_{FL} \cup S_{top-k}|$: how different the two selected sets are.
- **Facility-location gap** $FL(S_{FL}) - FL(S_{top-k})$: how much coverage value top- k leaves on the table.

Using the full-corpus pool here (rather than a condition-filtered one) is intentional: it ensures the filter is predictive of the evaluation outcome, since divergence is measured on the exact same candidate pool structure.

Output. `divergence_stats.parquet`: per-query divergence scores (`jaccard_div`, `fac_gap`, `fac_topk`, `fac_fl`) and a boolean `passes_filter` flag. Only passing queries proceed to gold annotation.

3d. Gold annotation (map-reduce LLM)

For each filtered query we need to know which chunks substantiate which answer facet. Facet labels are not pre-defined — the LLM generates them by inspecting the candidate pool. A key design requirement is *exhaustive tagging*: the LLM is instructed to tag *all* chunks that support

each facet, not just the best one. This is necessary for aspect recall to be a fair metric: a retrieved chunk contributes to a facet only if it appears in the gold set.

The candidate pool (top- N by cosine from admissions matching the query’s condition and modifier, so the annotation LLM sees clinically focused context) is processed in batches. For each batch the LLM produces (fact, facet_label, chunk_ids) triples. Facet labels from earlier batches are fed back into subsequent batch prompts to maintain a consistent vocabulary across the full pool. The reduce step merges results: each facet label maps to the union of all chunk IDs tagged under it across all batches. Outputs that fail schema validation (empty facts, hallucinated chunk IDs) are dropped.

Output. `gold_annotations.parquet`: one row per query with `facets_json` (mapping facet labels to lists of gold chunk IDs), plus `n_facets` and `n_gold_chunks` as summary fields. This is the ground truth consumed by Phase 4.

Phase 4 — Evaluation

Candidate pool construction

For each query the candidate pool is built by taking the top- N most similar chunks from the full corpus by cosine similarity, with no condition or modifier restriction. This is the deliberate choice: the pool should include the *redundant dominant cluster* (many very similar chunks about the primary condition) and the *complementary clusters* (chunks from patients with different comorbidities or demographics) — exactly the structure described in the introduction as necessary for coverage to outperform dispersion. Restricting the pool to condition-matched chunks would remove the dominant-cluster competition that makes the comparison interesting.

A pairwise cosine similarity matrix over the pool is computed once per query and shared across all strategies, since MMR, gMMR, and facility-location all require inter-chunk similarities.

Retrieval strategies

Four strategies are compared at each combination of k and λ :

- **top- k :** greedy similarity ranking. The non-diverse baseline, expected to collapse to the dominant cluster.
- **MMR** and **gMMR:** dispersion-based diversity. Penalize each new selection by its maximum similarity to already-selected chunks, spreading picks across the pool.
- **Facility-location:** coverage-based diversity. Greedy maximization of the submodular objective $g(S) = \sum_{i \in D} \max_{j \in S} \text{sim}(i, j)$, which ensures every region of the pool has a nearby representative.

Metrics

- **Aspect Recall (AR)** — primary metric: $|\{f \in F : S \cap G_f \neq \emptyset\}|/|F|$, the fraction of gold facets for which at least one gold chunk was retrieved. Directly measures multi-aspect coverage of the answer.
- **Weighted AR:** average per-facet recall weighted by facet size — penalizes retrieving only one of many gold chunks for a facet.
- **Gold Precision / Gold Recall:** standard precision and recall against the full gold chunk set.
- **FL coverage score:** the facility-location objective value $g(S)$ of the retrieved set — a geometry-based measure of how well the selection represents the candidate pool, independent of the gold annotations.

- **Jaccard vs. top- k :** set overlap between each method and the top- k baseline at the same k .
Confirms that methods rated highly by AR are also selecting differently from the baseline.

Outputs. `evaluation_results.parquet`: one row per (query, strategy, k , λ) combination with all metric values. `evaluation_stats.parquet`: means aggregated by strategy, forming the summary table for the thesis.